

CSIGen Documentation

Berkay Guler

Version 0.3.2

Last Updated: October 9th, 2026

1 Scene File Generation

The Geo2SigMap package provides scripts to extract an XML file containing scene information (buildings and ground terrain) from selected bounding box coordinates.

For example, the following command generates the file for the specified bounding box.

```
scenegen bbox -71.0602 42.3512 -71.0484 42.3591 --data-dir
scenes/sample_scene --enable-lidar-terrain
```

Note: bboxfinder.com is a simple website for obtaining the bounding box coordinates of a desired region. It can also be used to visualize a bounding box, given its coordinates.

If the `--enable-lidar-terrain` flag is not passed, the generated scene does not have elevation information, and a simple horizontal plane is used as the ground terrain.

The generated scene files are saved under the `--data-dir` folder, with the `scene.xml` file containing references to mesh objects stored as `.ply` files under the `mesh` subfolder. Load the scene using `load_scene(scene.xml, merge_shapes=False)`. Setting `merge_shapes=True` prevents objects from loading properly.

After loading the scene, access objects using `scene.objects["object_name"]`. Three objects are particularly important:

- `building_x.rooftop`: Rooftop object of the x th building
- `building_x.wall`: Wall object of the x th building
- `ground`: The terrain object from `lidar.terrain.ply`

Scene metadata (longitude/latitude bounding box values) is also stored in `scene.xml`.

2 Channel Generation

This framework enables generation of large-scale channel state information (CSI) datasets with customizable simulation parameters via the following configuration file.

```
config.yaml
1 # Example configuration for channel generation
2
3 # Scene configuration
4 scene_xml_path: "./scenes/boston_1/scene.xml" # Path to the 3D scene XML file generated by scenegen
5 carrier_frequency: 3500000000.0 # Carrier frequency in Hz
6
7 # Scene setup parameters
8 scene_center: [0.0, 0.0] # 2D coordinate [x, y] specifying scene center in meters
9 antenna_height_offset: 10.0 # Vertical offset of base station antennas above rooftops (meters)
10 num_deployment_buildings: 1 # Number of buildings for base station deployment
11 clip_terrain_to_buildings: true # Whether to clip terrain mesh to building boundaries
12 terrain_clip_margin: 15.0 # Margin around buildings when clipping (meters)
13 user_shift_from_ground: 1.5 # UE height above ground (meters)
14 # override_ground_material: "asphalt_concrete" # optional: "concrete" or "asphalt_concrete" (omit key if
    ↳ unused)
15
16 # TX antenna array parameters
17 tx_num_rows: 4 # Array dimensions (rows)
18 tx_num_cols: 8 # Array dimensions (columns)
19 tx_vertical_spacing: 0.5 # Element spacing in wavelengths (vertical)
20 tx_horizontal_spacing: 0.5 # Element spacing in wavelengths (horizontal)
21 tx_pattern: "tr38901" # Radiation pattern: tr38901 (3GPP), dipole, iso, or hw_dipole
22 tx_polarization: "V" # Polarization: cross (±45°), V, H, or VH
23
24 # RX antenna array parameters
25 rx_num_rows: 1 # Array dimensions (rows)
26 rx_num_cols: 1 # Array dimensions (columns)
27 rx_vertical_spacing: 0.5 # Element spacing in wavelengths (vertical)
28 rx_horizontal_spacing: 0.5 # Element spacing in wavelengths (horizontal)
29 rx_pattern: "iso" # Radiation pattern: tr38901 (3GPP), dipole, iso, or hw_dipole
30 rx_polarization: "V" # Polarization: cross (±45°), V, H, or VH
31
32 # Base station parameters
33 num_sectors: 6 # Sectors per base station (azimuth angles computed automatically for uniform coverage)
34 mechanical_tilt: 10.0 # Antenna downtilt angle (degrees)
35 azimuth_offset: 0.0 # Global azimuth rotation (degrees)
36 tx_power_dbm: 43.0 # Transmit power per array (dBm)
37
38 # Radio map solver parameters
39 radio_map_specular_reflection: true # Mirror-like reflections
40 radio_map_diffuse_reflection: true # Scattering from rough surfaces
41 radio_map_refraction: true # Transmission through materials (Snell's law)
42 radio_map_diffraction: true # Signal bending around obstacles
43 radio_map_edge_diffraction: true # Diffraction at edges
44 radio_map_diffraction_lit_region: false # Diffraction in LoS regions
45 radio_map_max_depth: 5 # Maximum ray-scene interactions per path
46 radio_map_samples_per_tx: 1000000 # Number of samples per TX antenna array (106)
47 radio_map_seed: 1 # Seed for radio map solver (reproducibility)
48
49 # User sampling parameters
50 num_user_samples_per_tx: 200 # Users sampled per TX array
51 user_sample_seed: 1 # Random seed for reproducibility
52 user_sample_min_val_db: -150 # Coverage thresholds (dB) - minimum
53 user_sample_max_val_db: 100 # Coverage thresholds (dB) - maximum
54 user_sample_min_dist: 0.0 # Distance constraints (meters) - minimum
55 user_sample_max_dist: 10000.0 # Distance constraints (meters) - maximum
56 user_sample_metric: "path_gain" # Sampling metric: path_gain, rss, or sinr
57 tx_association: true # Associate users with strongest-signal TX
58 sample_center_pos: true # Sample from cell centers (deterministic) vs. randomly within cells
59 scene_edge_epsilon: 1.0 # Minimum distance in meters from measurement surface edges to keep users (0.0
    ↳ means no filtering)
60
```

```

61 # Mobility preset
62 mobility_preset: "vehicular" # Select preset to use (must be a key in mobility_presets)
63
64 # User equipment mobility presets
65 mobility_presets:
66     stationary:
67         orientation_mode: "random" # random or to_tx (pointing toward serving TX)
68         speed_distribution: null # null (stationary), pedestrian (0.5--2 m/s), vehicular (8--30 m/s), or
        ↪ uniform_continuous with custom speed_min/speed_max
69     stationary_to_tx:
70         orientation_mode: "to_tx" # random or to_tx (pointing toward serving TX)
71         speed_distribution: null # null (stationary), pedestrian (0.5--2 m/s), vehicular (8--30 m/s), or
        ↪ uniform_continuous with custom speed_min/speed_max
72     pedestrian:
73         orientation_mode: "random" # random or to_tx (pointing toward serving TX)
74         speed_distribution: "pedestrian" # null (stationary), pedestrian (0.5--2 m/s), vehicular (8--30 m/s),
        ↪ or uniform_continuous with custom speed_min/speed_max
75         direction_mode: "random" # Movement direction (random)
76     pedestrian_to_tx:
77         orientation_mode: "to_tx" # random or to_tx (pointing toward serving TX)
78         speed_distribution: "pedestrian" # null (stationary), pedestrian (0.5--2 m/s), vehicular (8--30 m/s),
        ↪ or uniform_continuous with custom speed_min/speed_max
79         direction_mode: "random" # Movement direction (random)
80     vehicular:
81         orientation_mode: "random" # random or to_tx (pointing toward serving TX)
82         speed_distribution: "vehicular" # null (stationary), pedestrian (0.5--2 m/s), vehicular (8--30 m/s), or
        ↪ uniform_continuous with custom speed_min/speed_max
83         direction_mode: "random" # Movement direction (random)
84     slow_walking:
85         orientation_mode: "random" # random or to_tx (pointing toward serving TX)
86         speed_distribution: "uniform_continuous" # null (stationary), pedestrian (0.5--2 m/s), vehicular (8--30
        ↪ m/s), or uniform_continuous with custom speed_min/speed_max
87         speed_min: 0.5 # in m/s
88         speed_max: 1.5 # in m/s
89         direction_mode: "random" # Movement direction (random)
90     fast_walking:
91         orientation_mode: "random" # random or to_tx (pointing toward serving TX)
92         speed_distribution: "uniform_continuous" # null (stationary), pedestrian (0.5--2 m/s), vehicular (8--30
        ↪ m/s), or uniform_continuous with custom speed_min/speed_max
93         speed_min: 1.5 # in m/s
94         speed_max: 2.5 # in m/s
95         direction_mode: "random" # Movement direction (random)
96
97 # Path solver parameters
98 path_solver_max_depth: 5 # Maximum ray-scene interactions per path
99 path_solver_max_num_paths_per_src: 10000000 # Maximum paths per source (10^7)
100 path_solver_samples_per_src: 10000000 # Ray samples per source (10^7)
101 path_solver_synthetic_array: true # Use synthetic array approximation for efficiency
102 path_solver_los_mode: true # Include line-of-sight paths
103 path_solver_specular_reflection: true # Mirror-like reflections
104 path_solver_diffuse_reflection: true # Scattering from rough surfaces
105 path_solver_refraction: true # Transmission through materials (Snell's law)
106 path_solver_diffraction: true # Signal bending around obstacles
107 path_solver_edge_diffraction: true # Diffraction at edges
108 path_solver_diffraction_lit_region: false # Diffraction in LoS regions
109 path_solver_seed: 1 # Random seed
110 path_solver_per_tx_users_only: false # Compute paths only for TX-associated users
111 path_solver_rx_batch_size: 50 # Solve each TX's receivers in batches of this size (null = all at once)
112 path_solver_spec_table_size: 400000000 # Sionna specular-chain hash table size per source (null = Sionna
    ↪ default)
113
114 # OFDM parameters
115 num_subcarriers: 32 # OFDM subcarrier count
116 num_ofdm_symbols: 14 # Time-domain symbols
117 subcarrier_spacing: 30000.0 # Spacing in Hz
118
119 # Channel Frequency Response (CFR) parameters

```

```

120 cfr_normalize_delays: false # Normalize delays relative to first path
121 cfr_normalize: false # Normalize CFR magnitude
122 cfr_out_type: "numpy" # Output backend

```

2.1 Configuration Reference

2.1.1 Scene Configuration

- `scene_xml_path`: Path to the 3D scene XML file generated by `scenegen`.
- `carrier_frequency`: Carrier frequency in Hz.

2.1.2 Scene Setup Parameters

- `scene_center`: 2D coordinate $[x, y]$ specifying scene center in meters.
- `antenna_height_offset`: Vertical offset of base station antennas above rooftops (meters).
- `num_deployment_buildings`: Number of buildings for base station deployment.
- `clip_terrain_to_buildings`: Whether to clip terrain mesh to building boundaries.
- `terrain_clip_margin`: Margin around buildings when clipping (meters).
- `user_shift_from_ground`: UE height above ground (meters).
- `override_ground_material` (optional): ITU material type for the scene's `ground` object, `"concrete"` (1–100 GHz) or `"asphalt_concrete"` (1–40 GHz). The material is created if the scene does not define it, and wet-ground materials are removed. Omit the key or leave it unset to keep the scene XML materials; the ITU ground materials of the scene XML (e.g. `wet_ground`) are defined for 1–10 GHz only.

2.1.3 Antenna Array Parameters (TX and RX)

- `tx_num_rows, tx_num_cols / rx_num_rows, rx_num_cols`: Array dimensions.
- `tx_vertical_spacing, tx_horizontal_spacing / rx_vertical_spacing, rx_horizontal_spacing`: Element spacing in carrier wavelengths.
- `tx_pattern, rx_pattern`: Radiation pattern—`tr38901` (3GPP), `dipole`, `iso`, or `hw_dipole`.
- `tx_polarization, rx_polarization`: `cross` ($\pm 45^\circ$), `V`, `H`, or `VH`.

2.1.4 Base Station Parameters

- `num_sectors`: Sectors per base station (azimuth angles computed automatically for uniform coverage).
- `mechanical_tilt`: Antenna downtilt angle (degrees). Positive values tilt the boresight down.
- `azimuth_offset`: Global azimuth rotation (degrees).
- `tx_power_dbm`: Transmit power per array (dBm).

2.1.5 Ray Tracing Parameters (Radio Map and Path Solver)

These control ray-scene interactions for both the radio map and the path solver. The same physical toggles appear twice in YAML: each boolean is prefixed `radio_map_` for `RadioMapSolver` and `path_solver_` for `PathSolver`, so the two stages can be configured independently (e.g., fewer interactions or samples for the map, more for detailed paths). See [Sionna RT: Technical Report](#) for interaction semantics.

- `radio_map_max_depth`, `path_solver_max_depth`: Maximum ray-scene interactions per path.
- `radio_map_samples_per_tx`: Number of samples per TX for radio map (e.g., 10^6).
- `radio_map_seed`: Seed for radio map solver (reproducibility).
- `path_solver_max_num_paths_per_src`, `path_solver_samples_per_src`: Max paths and ray samples per source (e.g., 10^7).
- `radio_map_specular_reflection`, `path_solver_specular_reflection` (and `...diffuse_reflection`): Mirror-like and rough-surface scattering.
- `radio_map_refraction`, `path_solver_refraction`: Transmission through materials (Snell's law).
- `radio_map_diffraction`, `path_solver_diffraction`; `radio_map_edge_diffraction`, `path_solver_edge_diffraction`: Signal bending and edge diffraction.
- `radio_map_diffraction_lit_region`, `path_solver_diffraction_lit_region`: Diffraction in LoS regions.

2.1.6 User Sampling Parameters

- `num_user_samples_per_tx`: Maximum number of users sampled per TX array. Each user of a TX is placed in a different radio-map cell, so a TX with fewer valid cells gets fewer users, and a TX with no valid cell gets none. Each cell gets a random key from `user_sample_seed`, and a TX takes its valid cells with the lowest keys. The radio map is not bit-identical between GPU runs, so a few cells near the thresholds can change validity, and this way such a change moves at most one user.
- `user_sample_seed`: Random seed for reproducibility.
- `user_sample_min/max_val_db`: Coverage thresholds (dB).
- `user_sample_min/max_dist`: Distance constraints (meters).
- `user_sample_metric`: Sampling metric—`path_gain`, `rss`, or `sinr`.
- `tx_association`: If `true`, sampled positions are restricted to cells where the chosen metric is strongest for that transmitter; if `false`, a cell may be kept for one TX even if another TX would yield a higher metric at that location.
- `sample_center_pos`: If `true`, positions lie on radio-map cell centers (grid); if `false`, positions are drawn at random within each cell surface.
- `scene_edge_epsilon`: Minimum distance in meters from measurement surface outer edges to keep users. Users within this distance from any outer edge are filtered out. Set to 0.0 to disable filtering.

2.1.7 Mobility Presets

The top-level key `mobility_preset` must name one entry in the `mobility_presets` mapping. Each preset specifies:

- `orientation_mode`: `random` or `to_tx` (pointing toward serving TX).
- `speed_distribution`: `null` (stationary), `pedestrian` (0.5–2 m/s), `vehicular` (8–30 m/s), or `uniform_continuous` with custom `speed_min/speed_max`.

- `direction_mode`: Movement direction (`random`), required when the preset uses a non-null speed distribution.

2.1.8 Path Solver Parameters

Beyond the knobs listed here, path solving also uses `path_solver_max_depth` and the `path_solver_*` interaction toggles (specular reflection, diffuse reflection, refraction, diffraction, edge diffraction, diffraction-lit region). Those mirror the `radio_map_*` keys so the solver and radio map can diverge independently; see § 2.1.5.

- `path_solver_max_num_paths_per_src`: Maximum paths per source.
- `path_solver_samples_per_src`: Ray samples per source.
- `path_solver_synthetic_array`: Use synthetic array approximation for efficiency.
- `path_solver_los_mode`: Include line-of-sight paths.
- `path_solver_seed`: Random seed.
- `path_solver_per_tx_users_only`: Compute paths only for TX-associated users.
- `path_solver_rx_batch_size` (optional): Solve each TX's receivers (its own users or all users) in batches of this size. Sionna RT stores each new specular chain per receiver in a hash table of fixed size per source. When many receivers share one solve, the table fills up and new chains are discarded silently, so some NLoS paths are lost. Omit or set to `null` to solve all receivers at once.
- `path_solver_spec_table_size` (optional): Minimum size of the specular-chain hash table per source (Sionna RT's internal `SBCandidateGenerator.MIN_SPEC_COUNT_SIZE`; the table has $\max(\text{max_num_paths_per_src}, \text{this value})$ entries and takes 8 bytes per entry). Omit or set to `null` to keep the Sionna default. The table and the path buffer are indexed with 32-bit integers, so $\max(\text{max_num_paths_per_src}, \text{this value})$ times the number of sources must stay below 2^{32} ; the number of sources is 1 with a synthetic array and the number of TX antennas without, and the config validator rejects larger settings.

2.1.9 OFDM and CFR Parameters

- `num_subcarriers`: OFDM subcarrier count.
- `num_ofdm_symbols`: Time-domain symbols.
- `subcarrier_spacing`: Spacing in Hz.
- `cfr_normalize_delays`: Normalize delays relative to first path.
- `cfr_normalize`: Normalize CFR magnitude.
- `cfr_out_type`: Output tensor backend token (YAML string). Allowed values mirror `src.config_validator: numpy, tf` (TensorFlow), `torch`, `jax`, `drjit`.

3 Framework Architecture

The generation pipeline transforms a configuration file into channel frequency response (CFR) data through these sequential steps:

1. **Scene Setup:** Load 3D geometry, extract building info, optionally clip terrain, determine antenna positions.
2. **Antenna Configuration:** Configure shared TX/RX array properties.
3. **Base Station Deployment:** Deploy multi-sector base stations with uniform azimuthal coverage.
4. **Radio Map Computation:** Generate coverage maps via ray tracing.
5. **User Sampling:** Sample UE positions based on coverage, distance, and association constraints.
6. **Edge Filtering:** Optionally filter users too close to measurement surface edges (if `scene_edge_epsilon > 0`).
7. **Receiver Creation:** Create receivers with mobility parameters.
8. **Path Solving:** Compute ray-traced paths per TX.
9. **CFR Computation:** Convert paths to frequency response arrays (one tensor per TX over all receivers considered for that TX).
10. **Valid-Path Filtering and LoS/NLoS Labels:** After CFR is built, path delays and Sionna interaction types are evaluated in a single pass (`_compute_rx_valid_and_los_masks`). Receivers without any path with strictly positive delay ($\tau > 0$) are invalid and dropped. For each remaining (valid) channel, a binary line-of-sight flag is set: 1 if at least one valid path is pure LoS (all ray-scene interactions along that path are `InteractionType.NONE`), and 0 if all valid paths involve at least one non-trivial interaction (NLoS). The saved CFR tensor, `rx_positions`, and `rx_names` are sliced to valid channels only. Per-TX metadata includes the transmit antenna position (`tx_position`), receive positions for valid channels (`rx_positions`), corresponding receiver names (`rx_names`), `num_valid_channels`, `path_buffer_fill` (the largest fill of Sionna RT's per-source path buffer over the solves of the TX, as a fraction of `max_num_paths_per_src`; at 1 or above, paths were discarded, and a warning is logged), `users_per_tx` (the number of users sampled for each TX), `rx_serving_tx` (the TX each saved user was sampled for, aligned with the CFR user axis), and `los_binary` (length equal to the number of valid channels, aligned with the CFR user axis). It also records `cfr_axes`, the axes of the CFR tensor (`rx`, `rx_ant`, `tx`, `tx_ant`, `ofdm_symbol`, `subcarrier`), and `tx_array` and `rx_array`, the layout of each array: rows, columns, polarizations, spacing, pattern and the antenna index order $p * \text{num_rows} * \text{num_cols} + c * \text{num_rows} + r$, where r is the row counted from the top, c the column along the array's local y axis and p the polarization. The vertical index therefore runs fastest.

3.1 Key Design Choices

3.1.1 Shared Antenna Arrays

Sionna RT enforces a single TX and RX antenna configuration per scene. All base stations share identical TX array properties; all UEs share identical RX properties.

3.1.2 Per-Transmitter Path Solving

Paths are solved separately per TX rather than simultaneously for all TX-RX pairs. This reduces complexity from $O(N \times M)$ to $O(M)$ per TX. With TX association enabled, each TX processes only its associated users, reducing computation to $O(\text{num_users_per_tx})$ per TX. This enables scaling to scenarios infeasible with simultaneous solving.

3.1.3 Antenna Placement

Base stations deploy on the highest buildings (sorted by rooftop height). For each building, the antenna is placed at the rooftop vertex closest to the scene center, minimizing self-shadowing by positioning the building mass behind the antenna relative to the primary coverage area.

3.1.4 Measurement Surface

The framework creates a measurement surface by shifting the terrain mesh vertically by `user_shift_from_ground`. If terrain clipping is enabled, it occurs before this shift.

3.1.5 Configuration Validation

All parameters are validated before generation, catching errors early with clear messages. Use `load_validated_config(path)` (from `src.config_validator`) to load a YAML file into a dictionary suitable for the generator API.

3.1.6 Python API and Saved Outputs

Channel generation is exposed as `generate_channels(config)` in `src.channel_generator`: it is a *streaming* iterator that yields one dictionary per transmit sector index, in order, without holding all TX CFR tensors in memory at once. Each yielded item has keys `tx_idx`, `tx_name`, `h_tx` (NumPy CFR for valid channels only), and `tx_metadata` (positions, names, LoS flags, counts, embedded `config` copy, CFR shape/dtype, etc.).

The helper `save_channel_data(src.channel)` writes a compressed `.npz` per TX containing array `h`, `shape`, `dtype`, and a nested `metadata` object. The script `scripts/run.py` loads a config, iterates `generate_channels`, saves one file per TX under a timestamped output directory, and writes a run-level `metadata.yaml` summary (CSIGen version, scene name, timestamp, users per TX, per-TX CFR shapes/dtypes, CFR axes, array layouts, full config).

4 Train/Test Splits

A run stores one file per TX, and each row of a file is the channel between that TX and one user. In the all-users mode (`path_solver_per_tx_users_only: false`), each user therefore appears in the file of every TX. The script `scripts/split_dataset.py` splits a run into a train run and a test run by assigning every channel to train, to test or to neither, so that train and test share no users, no TXs, or neither, depending on the method:

```
python scripts/split_dataset.py --run output/<scene_name>/<run_id> \
    --out <out_dir> --by bs --test-ratio 0.2 --seed 1
```

4.1 Methods

- **users:** A random `test_ratio` of the user positions is test. All channels of a user are on the same side, so train and test share no users, but train users can lie next to test users.
- **area:** The bounding box of the user positions is tiled with square blocks of `--block-size` meters (default 50). Blocks are taken in random order and assigned to test until they hold at least `test_ratio` of the users, so the test share can exceed the ratio by up to one block (and all blocks but one stay train at most). With `--guard  $g > 0$` , train users within g meters (horizontal distance) of a test user are dropped, so the test users lie in areas without training users.
- **sector:** A random `test_ratio` of the sectors (TXs), or the sectors given with `--test-ids`, is test. All channels of a sector are on the same side. The users appear on both sides, with different sectors. A test sector usually has train sectors on the same base station, which share its position and see the same paths with another antenna orientation, so test results on a **sector** split are more optimistic than on a **bs** split.
- **bs:** As **sector**, for base stations with all their sectors. Base station b holds the TXs bN_s to $(b+1)N_s - 1$, where N_s is `num_sectors`.
- **sector+users, sector+area, bs+users, bs+area:** A TX split and a user split, each with `test_ratio`. A channel is train if both its TX and its user are train, test if both are test, and is dropped otherwise, so train and test share neither users nor TXs. In the all-users mode, test therefore keeps about r^2 of the channels and train about $(1-r)^2$ for `test_ratio` r ; choose a larger ratio for a usable test set.

With `--disjoint-users`, a **sector** or **bs** split also gives each user the side of the TX it was sampled for (`rx_serving_tx`) and drops the channels between a user and a TX of the other side. For **users**, **sector** and **bs**, the number of test items is the rounded `test_ratio` times the number of user positions, sectors or base stations, at least one and at most all but one; sectors and base stations without channels are not drawn. Users at the same position, which runs before CSIGen 0.3.0 can contain, are always kept on the same side: a **users** split draws positions rather than users, and in the other methods users at the same position that end up on different sides are dropped. All random choices use `--seed`, so the same command gives the same split.

4.2 Output

The output directory gets **train/** and **test/**, each a run in the usual format. Every TX file is written on both sides, restricted to the channels of that side, possibly none. The per-row metadata (`rx_names`, `rx_positions`, `los.binary`, `rx_serving_tx`) is restricted in the same way, `source.rows` holds the row index of each kept channel in the source file, and `split_side` names the side. Each side also gets a `split.yaml` with:

- the method, a description of what it holds out, and all parameters, including the seed;

- the test sectors, base stations or blocks that were chosen (blocks as indices $[i, j]$ of the tiling that starts at **block_origin**), the number of users dropped by the guard, and the number of users dropped because they share a position with a user on the other side;
- the source run (directory, scene, timestamp and CSIGen version) and the CSIGen version, git commit (with **-dirty** for uncommitted changes), command and time of the split;
- the number of channels and users on each side, the number of dropped channels, the number of users on both sides, and the number of channels of each TX on each side.

The user counts of the source run do not hold for a split run, so **users_per_tx** and **total_users** are renamed to **source_users_per_tx** and **source_total_users**, and **metadata.yaml** gets **num_users**, the number of users of the side. Runs made before CSIGen 0.3.0 have no **rx_serving_tx**; the script derives the serving TX from the user numbering.

5 Sample Flow

5.1 Scene Setup and Antenna Deployment

To avoid sampling users outside the urban area, terrain can be clipped at scene edges (Fig. 1). The highest `num_deployment_buildings` are selected for base station deployment (Fig. 2).

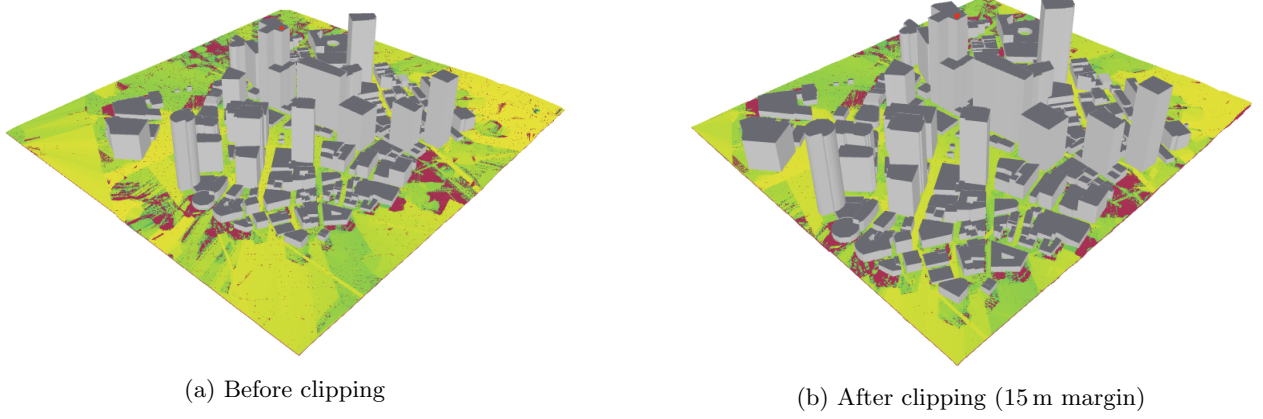


Figure 1: Effect of Terrain Clipping

5.2 Base Station Configuration

Each base station is deployed with `num_sectors` sectors at automatically computed azimuth angles for uniform coverage (e.g., 0° , 120° , 240° for 3 sectors). Sectors share position but differ in orientation.

5.3 Radio Map and User Sampling

The radio map computes signal strength across the measurement surface using shooting-and-bouncing-rays (SBR). User positions are sampled based on:

- Coverage thresholds (min/max signal strength)
- Distance constraints (min/max from TX)
- TX association (optional strongest-signal pairing)

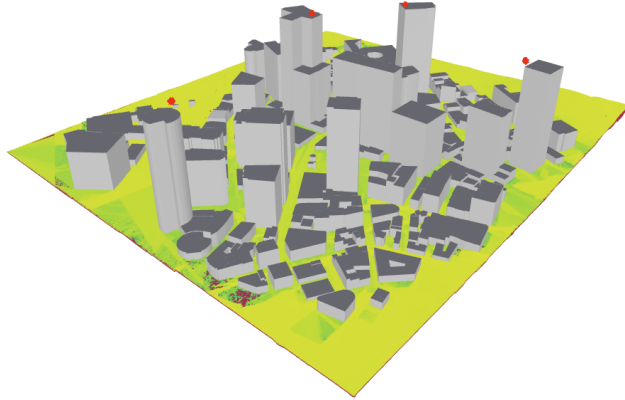


Figure 2: Base station deployment with `num_deployment_buildings=4` (red dots)

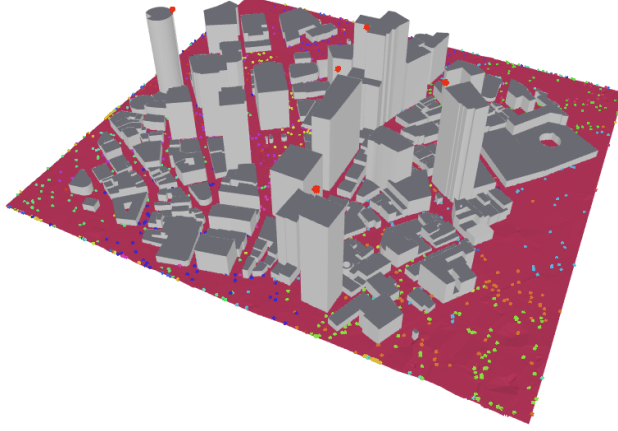


Figure 3: User sampling: 5 BSs $\times$ 3 sectors, 100 users/TX (colored points)

- Sampling mode (cell centers or random within cells)

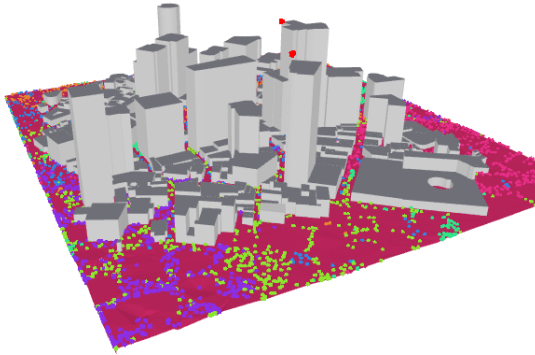
See Fig. 3 for an example with 5 base stations, 3 sectors each, and 100 users per TX.

5.4 Receiver Creation and Mobility

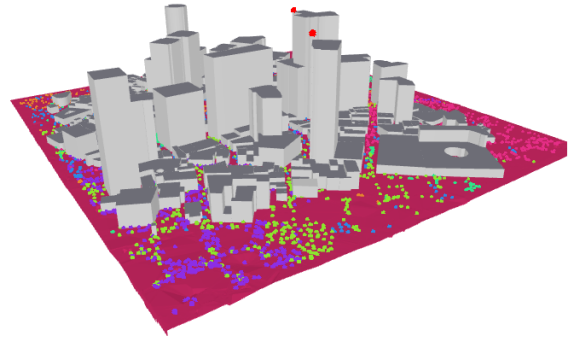
Receivers are created with mobility parameters from the selected preset. Orientation can be random or toward the serving TX. Velocity is sampled from the specified distribution with random direction.

5.5 Edge Filtering

After user sampling, positions can be filtered to remove users too close to the measurement surface outer edges. This is controlled by the `scene_edge_epsilon` parameter. The effect of this parameter is visualized in Fig. 4. Users at the far edges of the clipped measurement surface tend to be LoS-dominated, so this is a heuristic to remove them. Note that edge filtering is applied as a filter-out mechanism after user sampling. Therefore, it may change the number of effective users drastically especially if this parameter has a large value. Each TX keeps its own remaining users, so the number of users per TX can differ.



(a) `scene_edge_epsilon=0`



(b) `scene_edge_epsilon=15 meters`

Figure 4: Filtering out users close to outer edges of the scene. Here `num_sectors = 3`, `num_deployment_buildings = 2`, and `num_user_samples_per_tx = 1000`

5.6 Valid-Path Filtering and LoS/NLOS Labels

After path solving and CFR computation, the pipeline evaluates validity and LoS/NLOS jointly from the `Paths` object: path delays ($\tau > 0$) and per-path interaction records (`Sionna interactions`) determine which receivers have any valid path and whether a valid channel has at least one pure LoS path (no reflections, diffraction, etc.). Receivers with no valid paths are dropped; CFR arrays and aligned metadata (`rx_positions`, `rx_names`) are restricted to the valid subset. Per-TX metadata also stores the serving sector antenna position as `tx_position`. The field `num_valid_channels` gives the count of valid channels, and `los_binary` is a length-`num_valid_channels` array with 1 for LoS and 0 for NLoS, in the same order as rows of the filtered CFR. We include a histogram of number of paths from a TX to all sampled users displayed in Fig. 5.

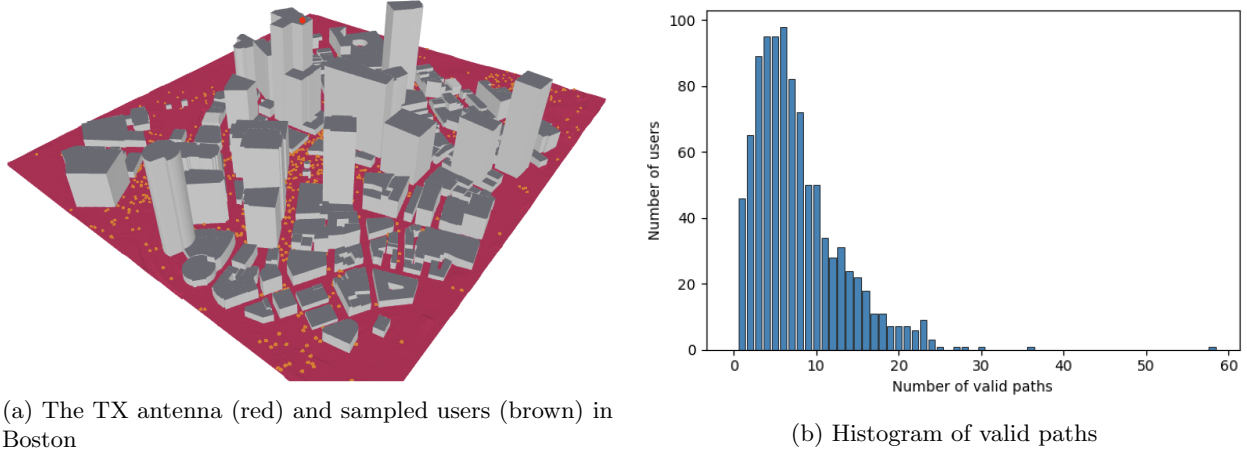


Figure 5: Distribution of number of paths after removing invalid users

5.7 Channel Frequency Response

After the paths are computed from a TX to a set of RXs, the channel frequency response can be derived from the path gains and delays. For `mobility_preset="vehicular"`, `num_subcarriers=32`, `tx_num_rows=1`, `tx_num_cols=32`, `rx_num_rows=1`, `rx_num_cols=1`, `num_ofdm_symbols=14`, and `subcarrier_spacing=30` KHz, we include the time-frequency response for a fixed TX antenna index in Fig. 6 and TX antenna-frequency response for a fixed OFDM symbol in Fig. 7.

5.8 Performance Optimizations

- **Synthetic Array:** Treats array as continuous aperture for faster computation.
- **Per-TX Solving:** Reduces complexity; enables large-scale generation.
- **TX Association Filtering:** Processes only associated users per TX.
- **Radio Map Pre-filtering:** Uses fewer samples (e.g., 10^6) than detailed path solving (e.g., 10^7) for efficient coverage estimation.

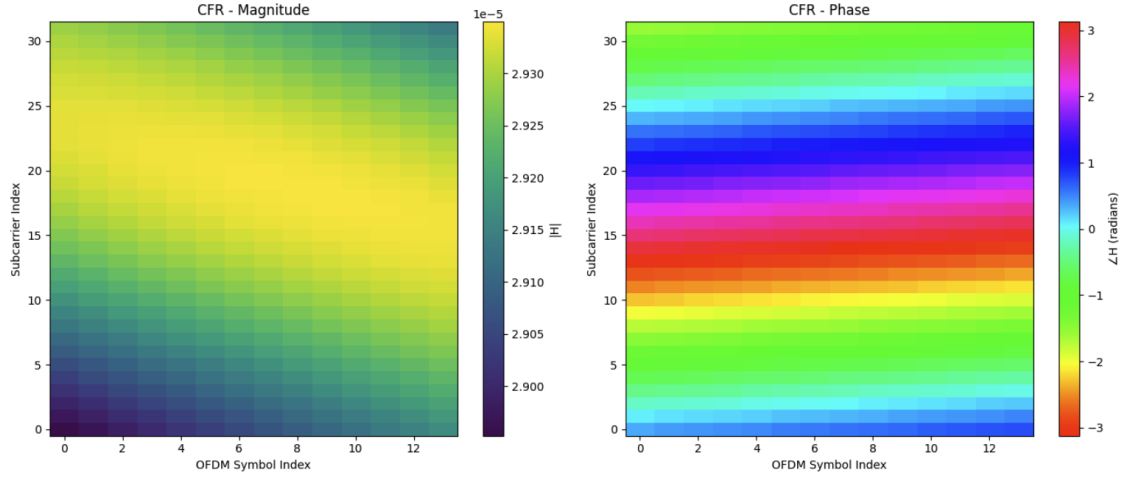


Figure 6: Time-frequency response (magnitude on the left, phase on the right)

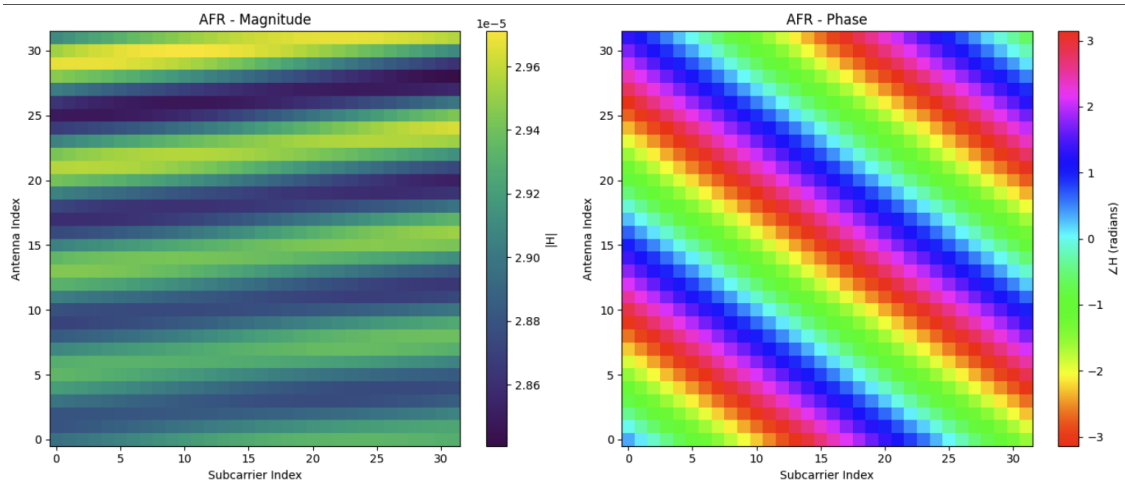


Figure 7: TX antenna-frequency response (magnitude on the left, phase on the right)

6 Available Scenes

We generated a number of scenes using `scenegen` tool. These scenes are included here.

6.1 Boston Downtown

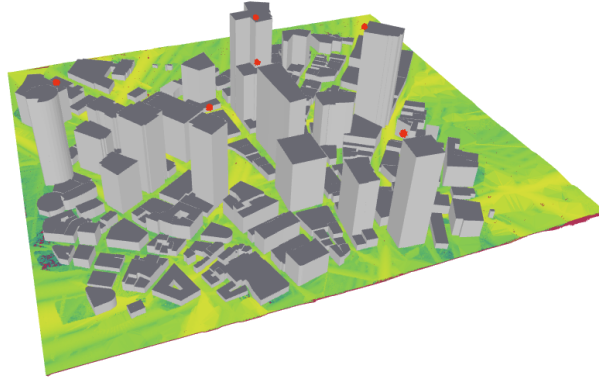


Figure 8: boston_1 scene

6.2 Chicago Downtown

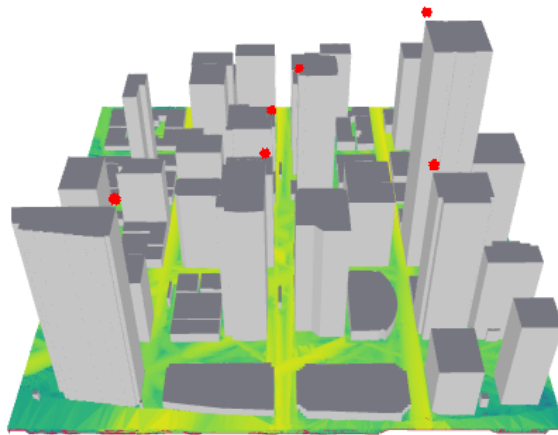


Figure 9: chicago_1 scene

6.3 New York City Manhattan

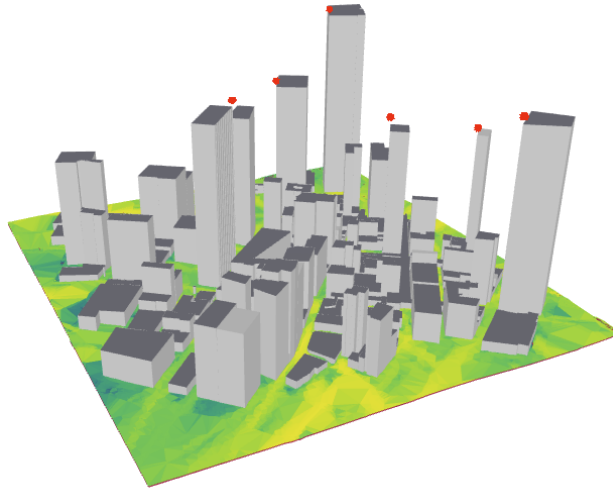


Figure 10: `nyc_1` scene

6.4 San Francisco Downtown

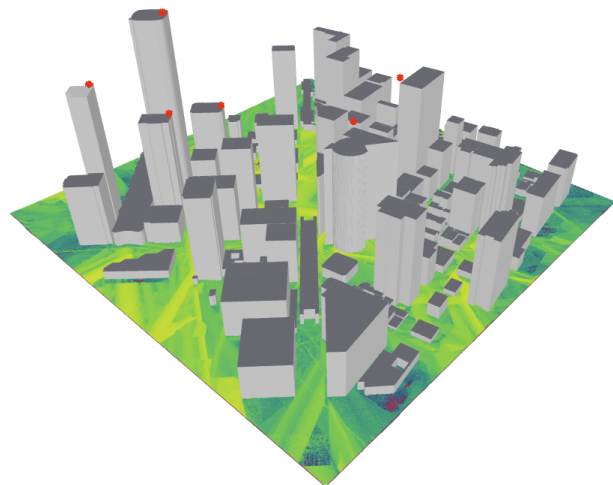


Figure 11: `sf_1` scene

6.5 Los Angeles Downtown

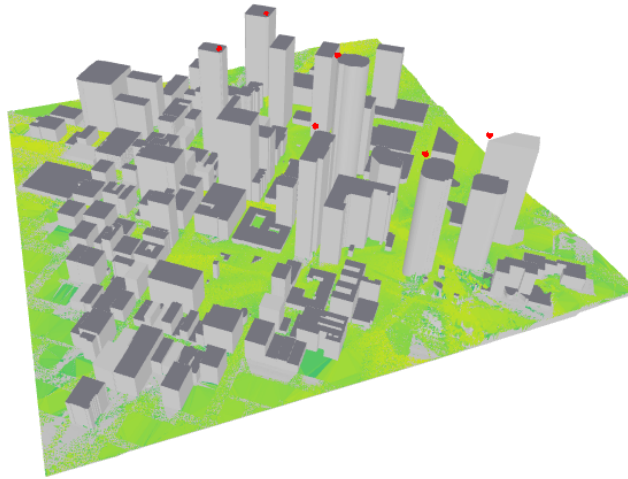


Figure 12: 1a.1 scene